



# Estruturas de Controle em Python

O Guia Visual para Lógica de Programação, Condicionais e Laços de Repetição

Uma abordagem passo a passo para o fluxo de execução.



# A Base da Lógica: Operadores

## Operadores de Comparação

| Operator           | Meaning        | Example                               |
|--------------------|----------------|---------------------------------------|
| <code>==</code>    | Igual a        | <code>3 == 3</code> → <b>True</b>     |
| <code>!=</code>    | Diferente de   | <code>2 != 3</code> → <b>True</b>     |
| <code>&lt;</code>  | Menor que      | <code>2 &lt; 3</code> → <b>True</b>   |
| <code>&gt;</code>  | Maior que      | <code>3 &gt; 2</code> → <b>True</b>   |
| <code>&lt;=</code> | Menor ou igual | <code>5 &lt;= 4</code> → <b>False</b> |
| <code>&gt;=</code> | Maior ou igual | <code>5 &gt;= 4</code> → <b>True</b>  |

## Operadores Lógicos

| Operator         | Meaning                                 | Example                                              |
|------------------|-----------------------------------------|------------------------------------------------------|
| <code>and</code> | Retorna True se ambos forem verdadeiros | <code>(3 &lt; 4) and (5 &gt; 4)</code> → <b>True</b> |
| <code>or</code>  | Retorna True se um for verdadeiro       | <code>(3 &lt; 4) or (3 &gt; 5)</code> → <b>True</b>  |
| <code>not</code> | Inverte o valor                         | <code>not 3 &lt; 2</code> → <b>True</b>              |

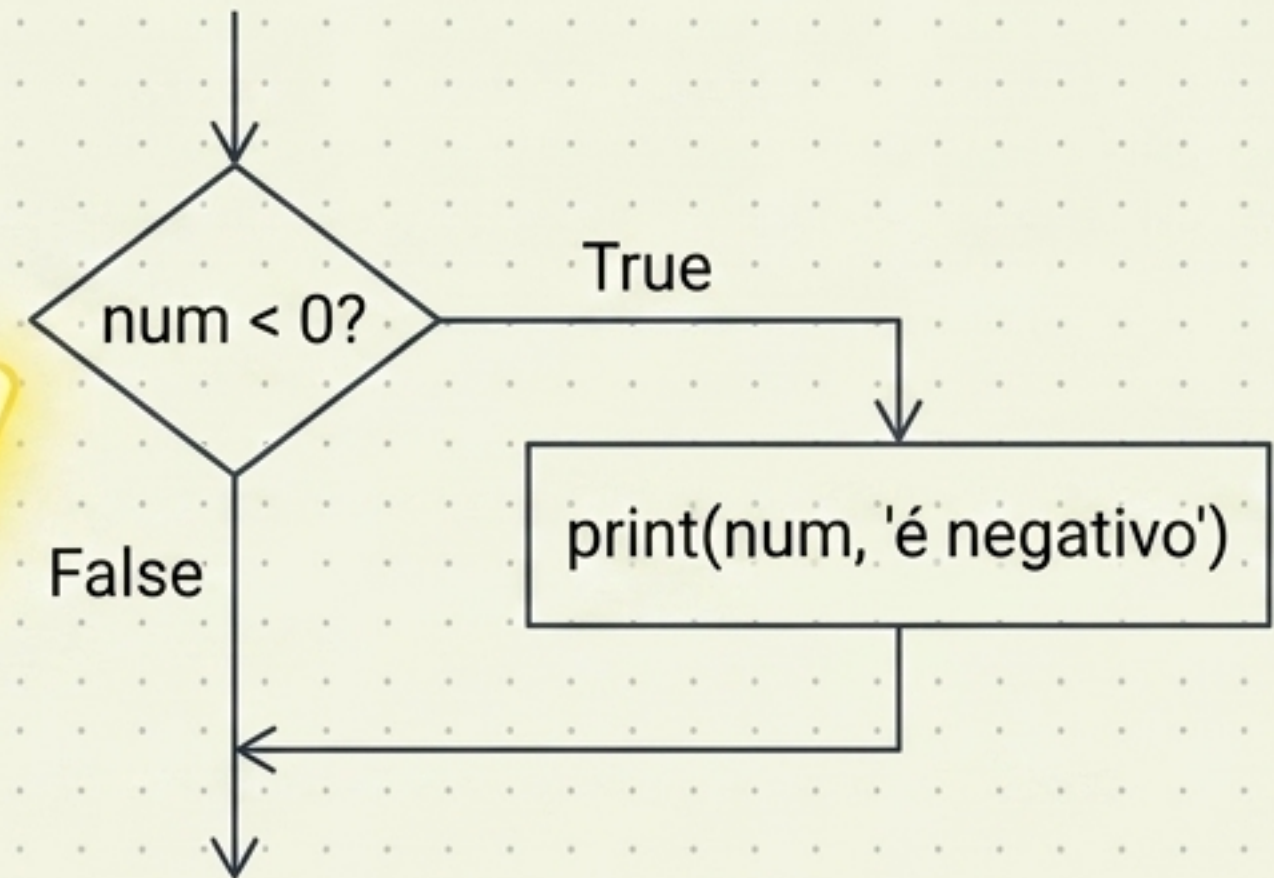
**A Regra de Ouro do Python:** Zero (0), strings vazias (""), e None são sempre avaliados como **False**. Qualquer outro valor ou objeto é avaliado como **True**.



# Ramificação Simples: O Comando if

```
num = int(input('Digite um número: '))
```

```
if num < 0:  
    print(num, 'é negativo')
```

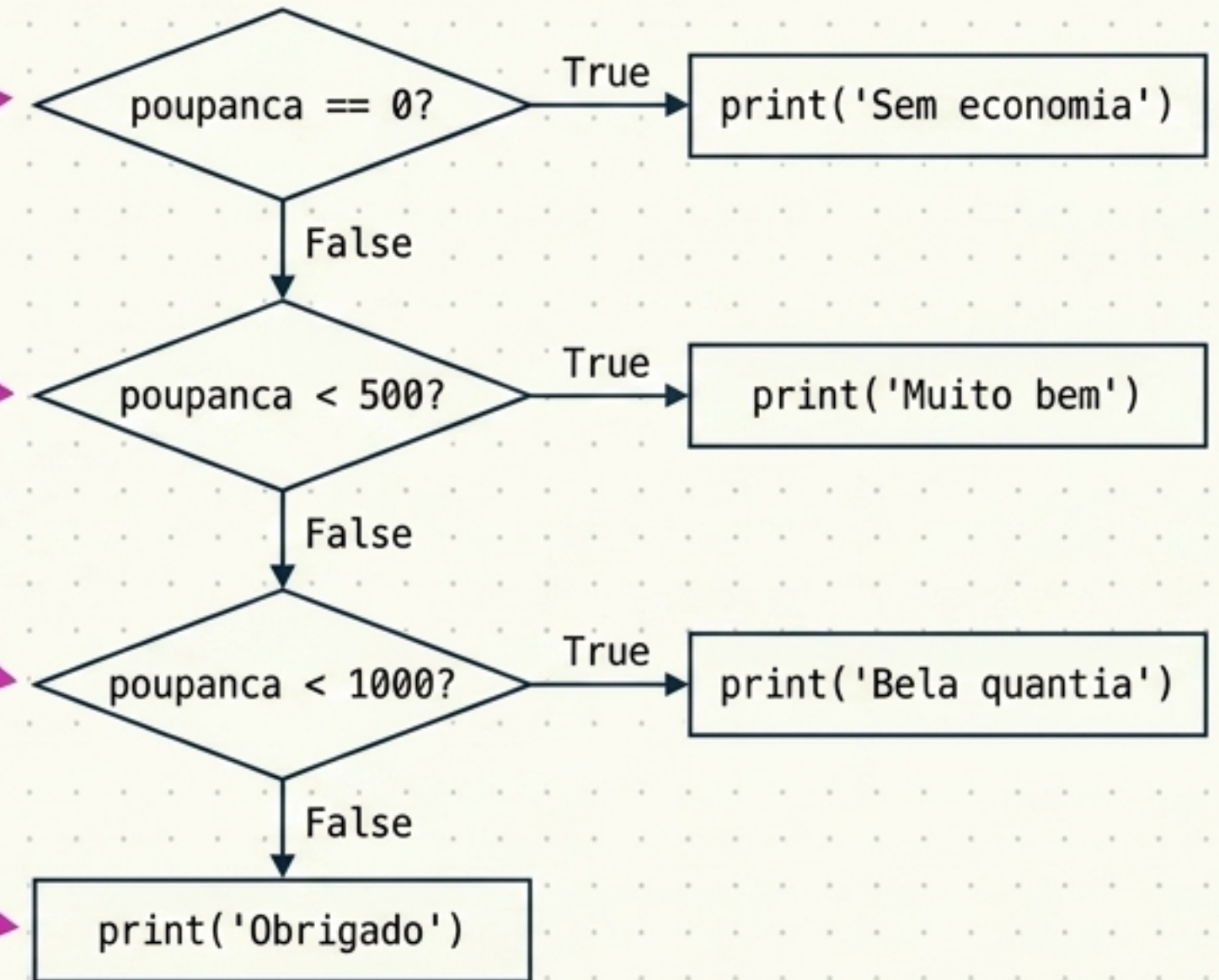


A indentação é essencial: determina qual bloco de código pertence à instrução **if**. O caractere **:** sinaliza o início desse bloco condicional.



# Múltiplos Caminhos: **elif** e **else**

```
if poupanca == 0:  
    print('Sem economia')  
elif poupanca < 500:  
    print('Muito bem')  
elif poupanca < 1000:  
    print('Bela quantia')  
else:  
    print('Obrigado')
```



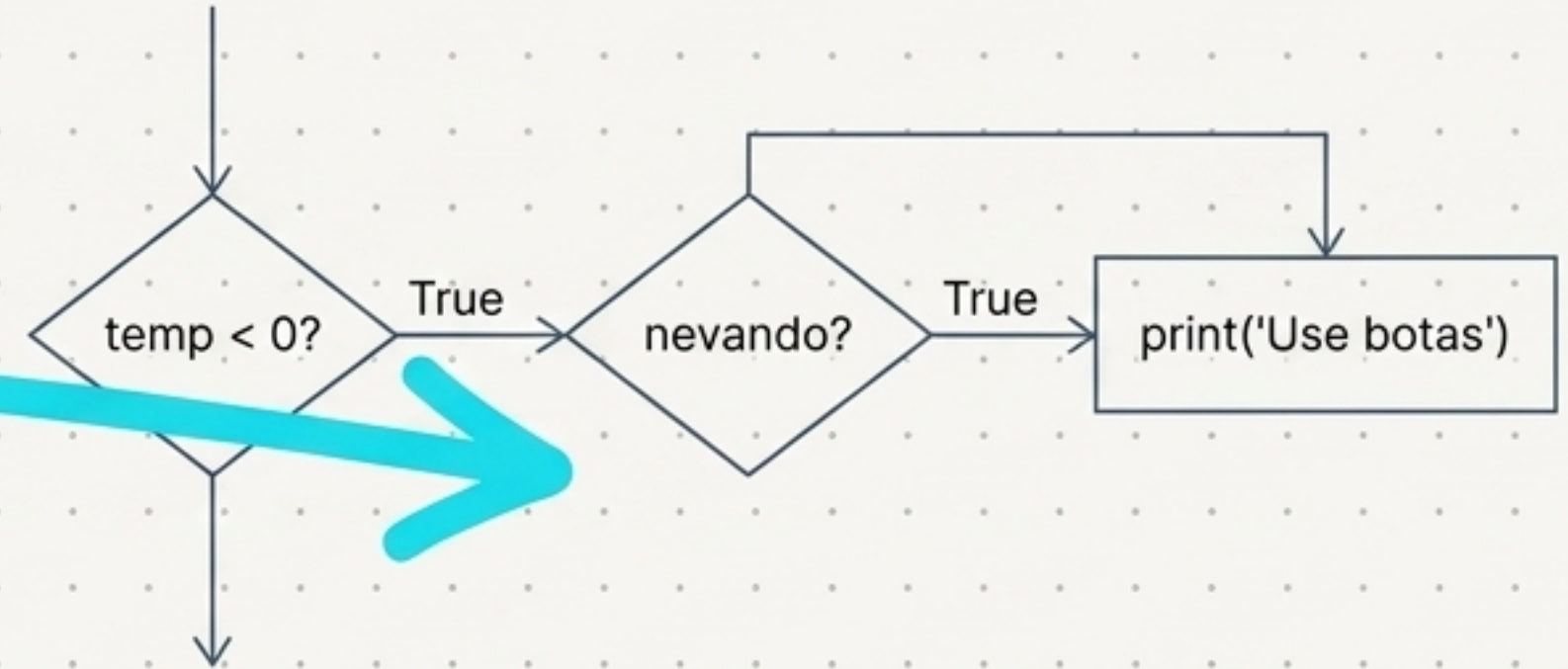
O Python avalia as condições de cima para baixo. Assim que uma condição retorna True, seu bloco correspondente é executado e o restante da estrutura é ignorado. O else final é opcional e garante que pelo menos uma ação seja executada.



# Lógica Avançada: Aninhamento e Expressões

## Aninhamento (Nesting)

```
if temp < 0:  
    if nevando:  
        print('Use botas')
```



## Expressões if (Operador Ternário)

resultado = ( **[Valor Se Verdadeiro]** **if** **[Condição]** **else** **[Valor Se Falso]** )

```
status = 'adolescente' if age > 12 and age < 20 else 'não adolescente'
```

Uma forma concisa de atribuir valores. Retorna o primeiro valor a menos que a condição falhe.



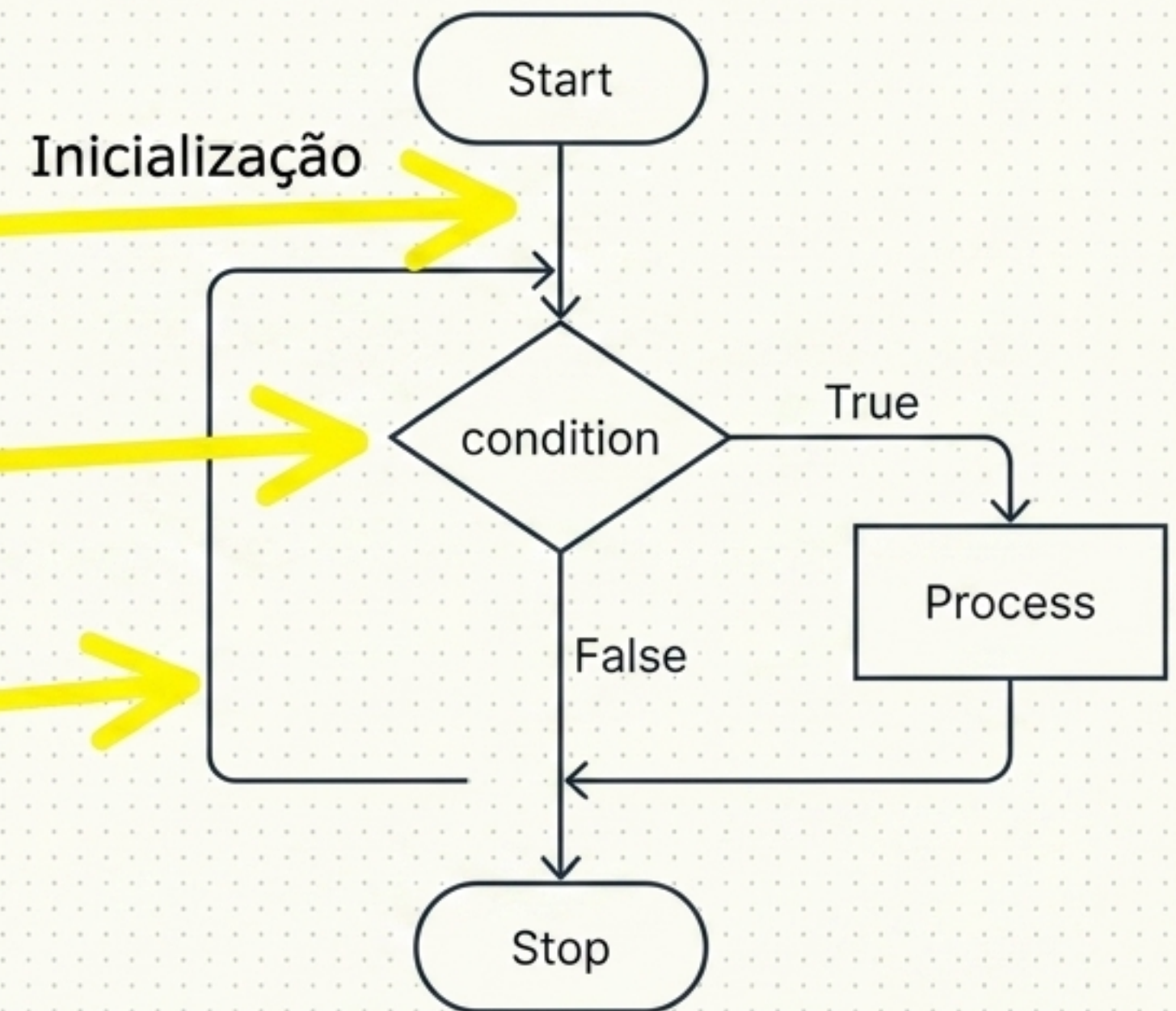
# O Ciclo Infinito: Laços while

```
count = 0
```

```
while count < 10:
```

```
    print(count)
```

```
    count += 1
```

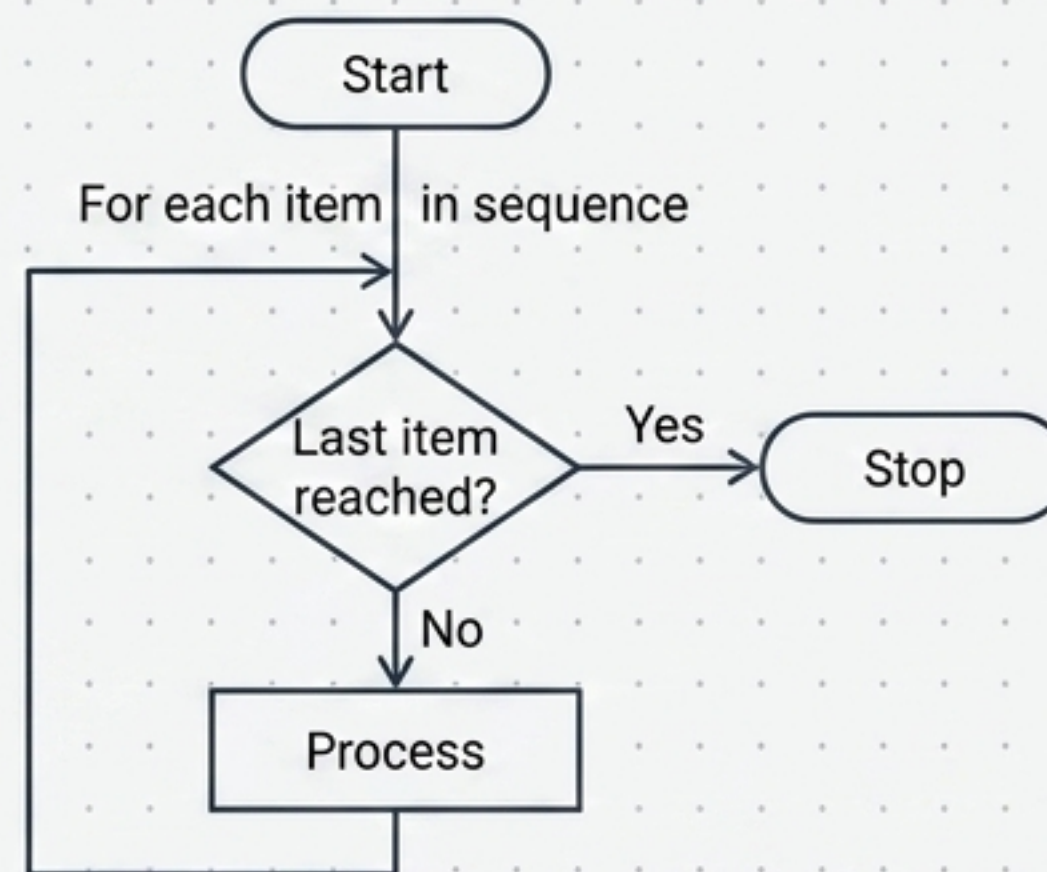


O laço iterará enquanto a condição for True. A verificação ocorre antes de cada iteração. Se falhar na primeira vez, o bloco nunca será executado.



```
for i in range(0, 10, 2):  
    print(i)
```

## Iteração Definida: Laços for e a Função range()



O laço for é mais conciso. Ele percorre automaticamente uma série de valores até que o último item seja alcançado, sem necessidade de definir ou incrementar a variável de controle manualmente.

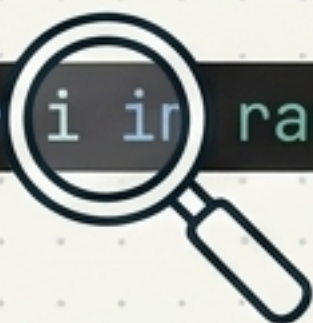


# Convenções de Repetição: Nomeando Variáveis

## A Tradição (i, j)

Em iteradores matemáticos ou baseados em intervalo (range), use i e j. Essa convenção universal remonta à linguagem Fortran (década de 1950).

```
for i in range(5):
```



## Nomes Descritivos

Para listas de objetos (vistas em capítulos futuros), use o singular do item real.

```
for aluno in alunos:
```



## O Curinga Anônimo (\_)

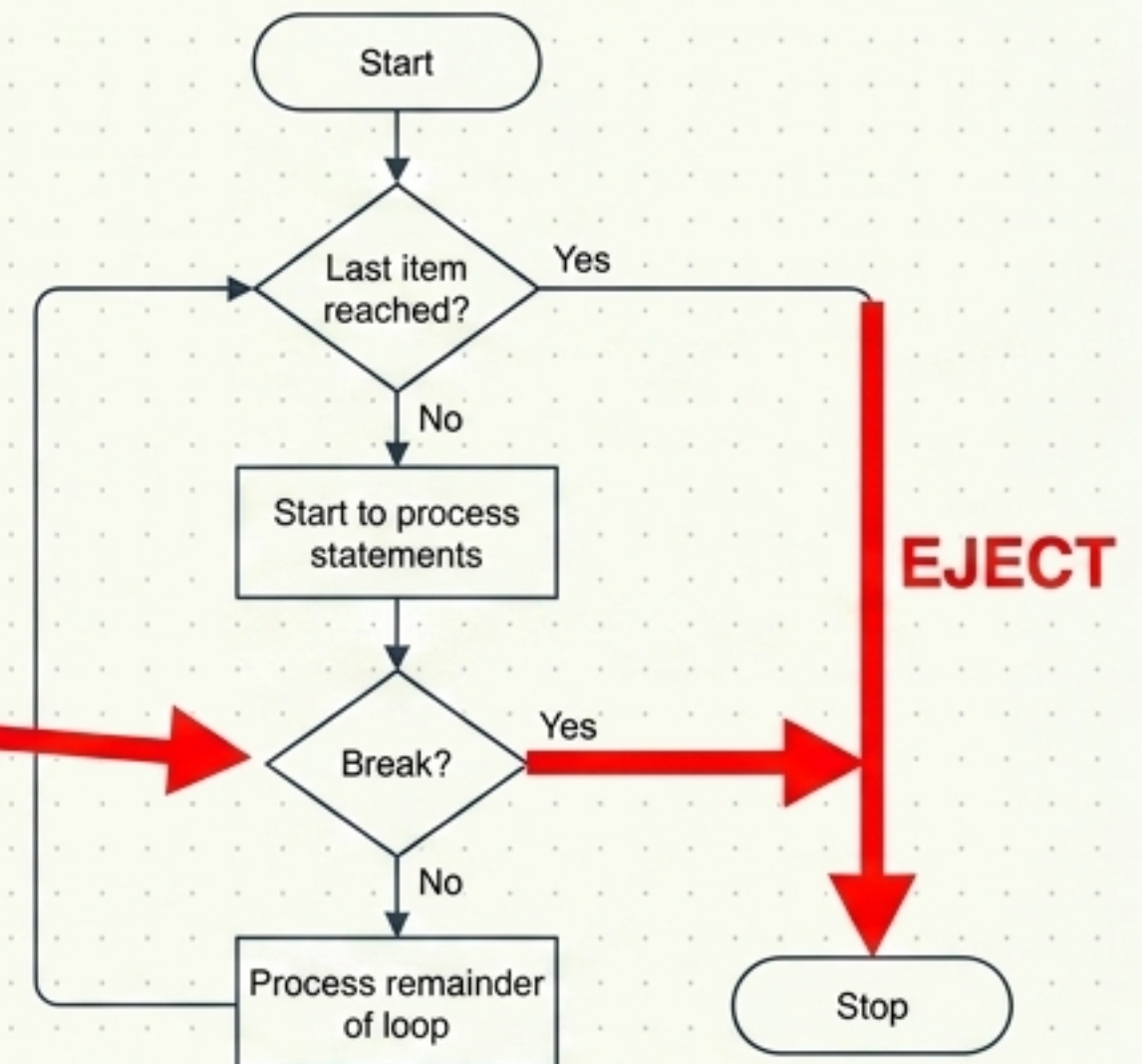
```
for _ in range(10):  
    print('.')
```

Use o caractere underline (\_) quando você precisar executar o ciclo um número específico de vezes, mas não tiver interesse no valor numérico da iteração.



```
for i in range(0, 6):  
    if i == num:  
        break  
    print(i)
```

# Interrompendo o Fluxo: O Comando break

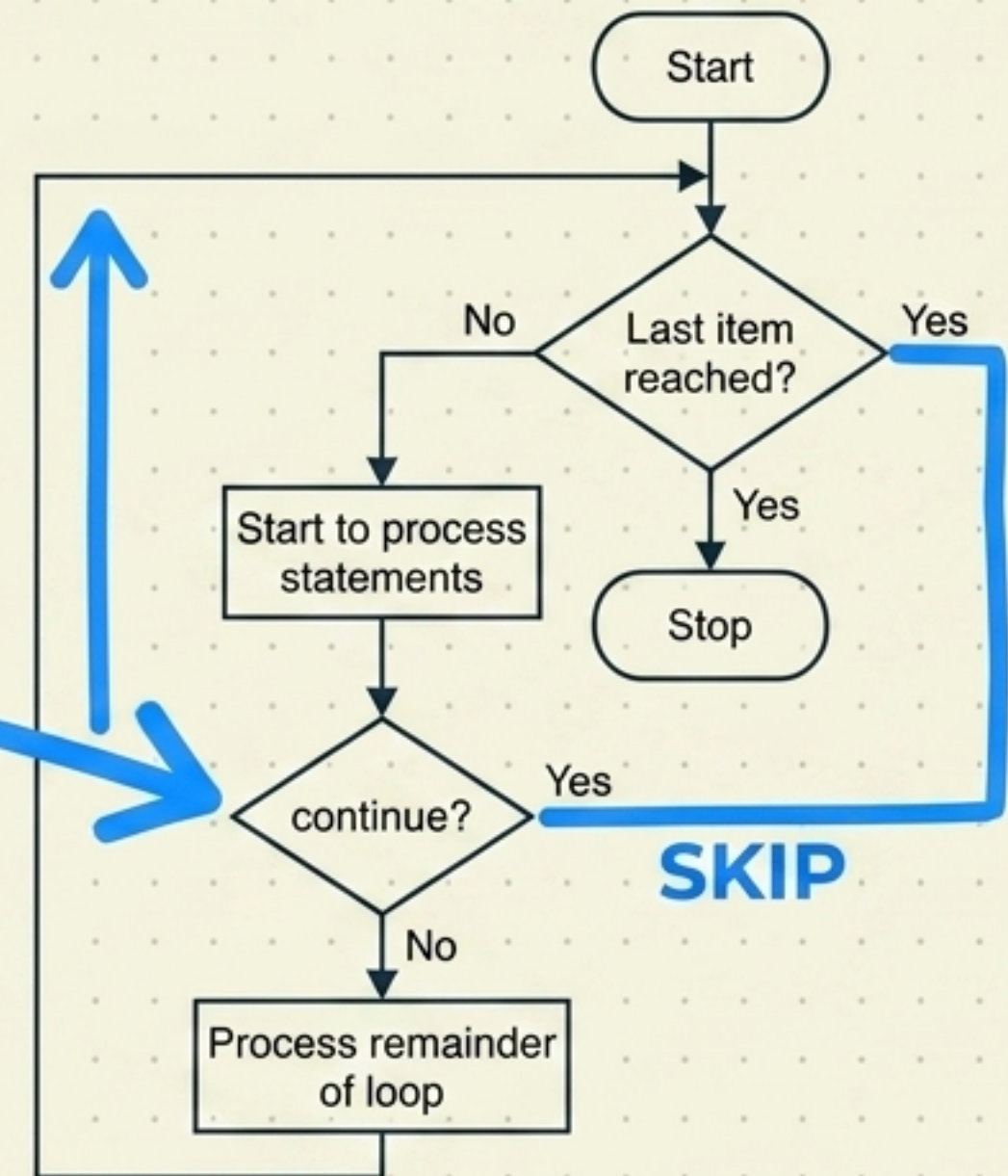


Geralmente protegido por uma instrução if, o break altera o ciclo normal, terminando o laço imediatamente e saltando para a primeira linha após o bloco de repetição.



# Pulando Etapas: O Comando continue

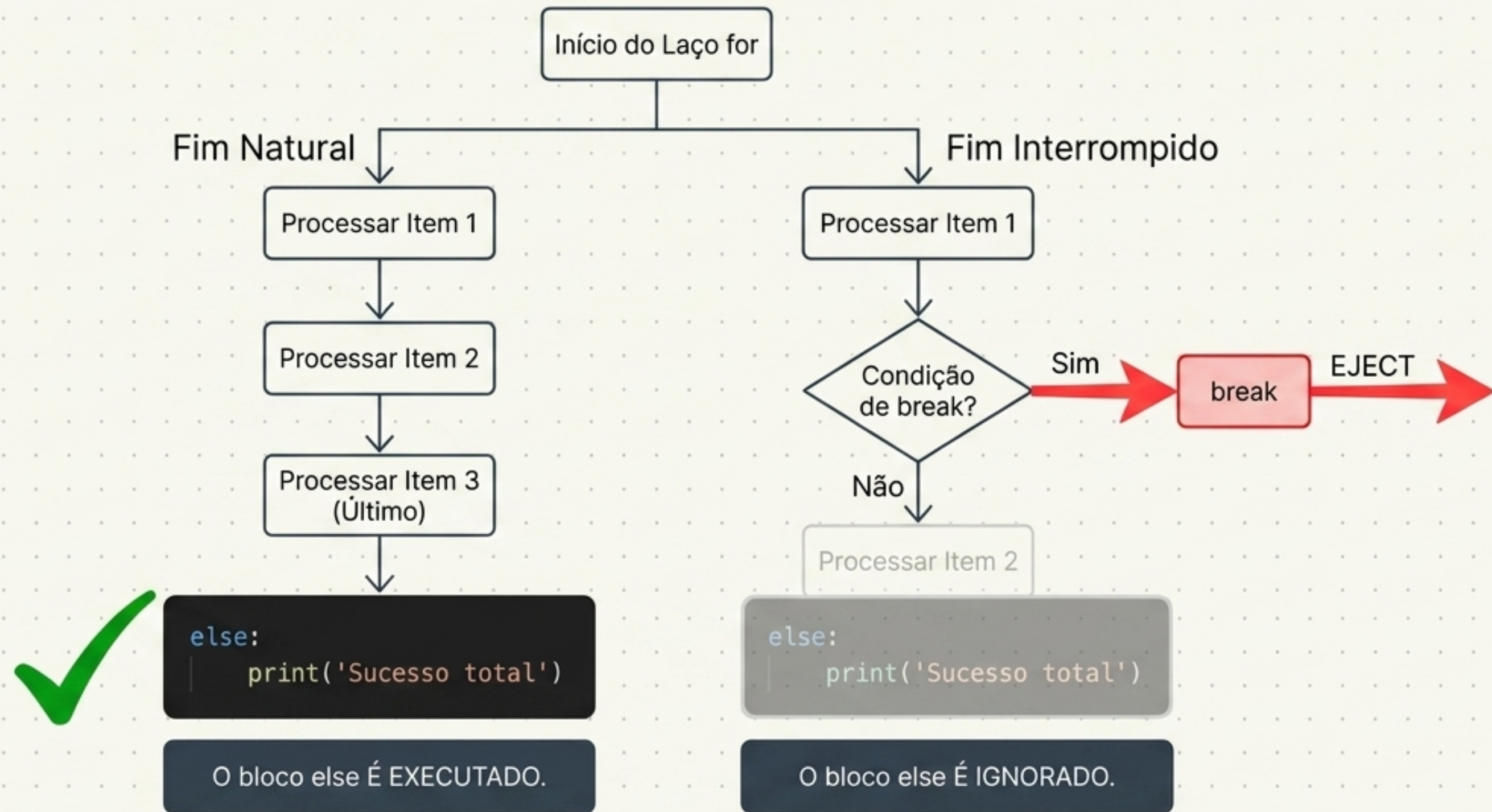
```
for i in range(0, 10):  
    if i % 2 == 1:  
        continue  
    print('Par:', i)
```



Não encerra o laço inteiro. Apenas aborta a iteração atual e retorna imediatamente para avaliar o próximo valor da sequência.



# O Caso Especial de Python: for com else



O bloco else no final de um laço é executado apenas se todos os itens da sequência forem processados sem que um break seja acionado.



# Síntese Diagnóstica: **for** vs. **while**

|                        | Laço for                                                                                        | Laço while                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Quando usar            | Iterações Conhecidas<br>(percorrer um intervalo numérico ou coleção)                            | Iterações Desconhecidas<br>(repetir até que um evento ocorra, ex: input do usuário)        |
| Complexidade Sintática | Alta densidade, variável de controle é automática.<br>(ex: <code>`for i in range(10):`</code> ) | Menor densidade, exige inicialização e incremento manuais.<br>(ex: <code>`i += 1`</code> ) |
| Risco de Loop Infinito | Muito Baixo (sequências sempre têm um fim)                                                      | Alto (se o programador esquecer de atualizar a variável de controle)                       |
| Caso de Uso Ideal      | "Processe estes 10 itens."                                                                      | Jogo de dados: "Continue rolando até o usuário digitar 'N'."                               |



# O Fluxo de Execução em Resumo

## 1. Avaliação

Matriz Booleana  
(True/False)



## 2. Decisão

Condicionais  
(if/elif/else)



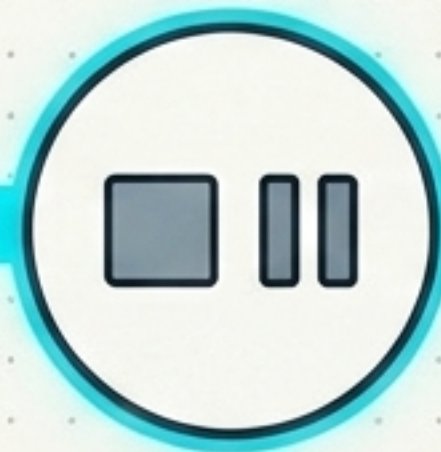
## 3. Repetição

Laços (for/while)



## 4. Intervenção

Controle de Laços  
(break/continue)



**Estruturas de controle convertem código estático em sistemas dinâmicos e decisórios. O domínio destas regras de indentação e lógica é a base de todo programa Python.**